# Week 09

## NumPy와 배열 사고

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

# 중간 실기 돌아보기

| 흔한 오류 | 원인 | 다시 보기 |
|---|---|---|
| `"80" * 0.3` | CSV 값은 문자열 | 6주차 |
| 출석 미달인데 A | 규정 확인 순서 | 3주차 |
| `distribution`에 0명 등급 누락 | 키를 미리 만들지 않음 | 4주차 |
| 원본 리스트 정렬 | `sort()` 사용 | 4주차 |

> 오늘부터는 같은 계산을 **반복문 없이, 배열 전체에 한 번에** 한다

# 오늘의 목표

1. 반복문 대신 **배열 전체 연산**하는 "배열 사고"
2. 생성 · `shape` · `dtype` · 인덱싱 · **불리언 마스크** · `np.where`
3. **axis**로 행 / 열 방향 집계, **NaN** 안전하게 다루기
4. 1년치 기온 데이터로 월별 요약 · 이동평균 · 이상값
5. AI 코드의 **axis 혼동**을 shape와 작은 예제로 찾기

# 수업 흐름

| 구간 | 시간 | 활동 |
|---|---|---|
| 도입 · 개념 | 20분 | 리스트 vs 배열, shape, axis |
| 안내 실습 | 45분 | 생성 · 연산 · 마스크 · 집계 · NaN |
| 독립 · 짝 실습 | 35분 | 1년치 기온 데이터 분석 |
| 공유 · 점검 | 20분 | axis 혼동 사례, 과제 안내 |

## 이번 주 Colab 실습 열기

# 리스트 vs NumPy 배열

```
[1, 2, 3] + [10, 20, 30]                          # [1, 2, 3, 10, 20, 30]
np.array([1, 2, 3]) + np.array([10, 20, 30])  # [11 22 33]
[1, 2, 3] * 2                                      # [1, 2, 3, 1, 2, 3]
np.array([1, 2, 3]) * 2                           # [2 4 6]
```

| | 리스트 | 배열 |
|---|---|---|
| 자료형 | 섞여도 됨 | 한 가지 (`dtype`) |
| 연산 | 반복문 | **벡터화** (전체에 한 번에) |
| 속도 | 느림 | 수십~수백 배 빠름 |

# 배열 만들기

```
a = np.array([72, 95, 88, 59, 100])
a.dtype, a.shape, a.ndim, a.size      # int64, (5,), 1, 5

np.arange(0, 10, 2)         # [0 2 4 6 8]
np.linspace(0, 1, 5)        # [0.   0.25 0.5  0.75 1.  ]
np.zeros(3), np.ones((2, 3))

rng = np.random.default_rng(42)       # 시드 고정 → 재현 가능
rng.integers(0, 101, size=5)
rng.normal(70, 10, size=3)
```

# 2차원 배열: 학생 × 과목

```
scores = np.array([
    [78, 85, 90],      # 학생 0: 중간, 기말, 과제
    [92, 88, 95],
    [65, 70, 72],
    [88, 91, 84],
    [55, 62, 70],
])
scores.shape           # (5, 3) = (행, 열) = (학생, 과목)
```

> 배열을 다룰 때는 항상 **"각 축이 무엇인가?"** 부터 말로 정한다

# 브로드캐스팅

```
weights = np.array([0.3, 0.4, 0.3])      # shape (3,)
weighted = scores * weights              # (5,3) * (3,) → (5,3)
total = weighted.sum(axis=1)             # 학생마다 합 → (5,)
total = scores @ weights                 # 같은 결과 (행렬 곱)

np.minimum(scores + 5, 100)              # 5점 가산, 100점 상한
np.clip(scores - 10, 0, 100)             # 0~100으로 자르기
```

모양이 달라도 **끝 차원이 맞으면** 자동으로 늘려 계산

# 인덱싱과 슬라이싱

| 표현 | 의미 |
|---|---|
| `scores[1]` | 1번 학생 (행) |
| `scores[:, 0]` | 모든 학생의 0번 과목 (열) |
| `scores[1, 2]` | 1번 학생의 2번 과목 |
| `scores[1:3, :2]` | 1~2번 학생, 0~1번 과목 |

`[행, 열]` — 콤마 앞이 행, 뒤가 열

# 불리언 마스크

```
mask = total >= 80                    # [ True  True False  True False]
total[mask]                           # 80점 이상 총점만
mask.sum()                            # 인원 (True = 1)
names[mask]                           # 이름 고르기

# 조건 합치기: and/or 대신 & | 와 괄호
((scores[:, 0] < 60) | (scores[:, 1] < 70)).sum()
temps[~np.isnan(temps)]               # ~ 는 뒤집기
```

`a >= 60 and a < 90` → **오류** / `(a >= 60) & (a < 90)` → OK

# np.where · np.select

```
np.where(total >= 70, "통과", "재수강")

np.select(
    [total >= 90, total >= 80, total >= 70],
    ["A", "B", "C"],
    default="F",
)
```

| 함수 | 용도 |
|---|---|
| `np.where(조건, 참값, 거짓값)` | 둘 중 하나 고르기 |
| `np.where(조건)` | 참인 **위치** 찾기 |
| `np.select(조건들, 값들, default)` | 여러 단계 고르기 |

# 예상해 보기: axis

```
scores.shape                  # (5, 3)
scores.mean(axis=0).shape     # ?
scores.mean(axis=1).shape     # ?
```

정답: `(3,)` 과목별 · `(5,)` 학생별

> **axis = 사라지는 방향**
> `axis=0` 위→아래로 접기 → 열마다 하나 / `axis=1` 왼→오로 접기 → 행마다 하나

# 집계 함수

```
scores.max(axis=0)           # 과목별 최고점
scores.argmax(axis=0)        # 과목별 최고점 학생 번호
scores.argmin(axis=1)        # 학생별 가장 낮은 과목
scores.std(axis=0)           # 과목별 표준편차
np.cumsum([1, 2, 3, 4])      # [ 1  3  6 10]
np.percentile(total, [25, 50, 75])
(scores >= 60).all(axis=1)  # 모든 과목 60 이상인 학생
```

# 결측값 NaN

```
temps = np.array([12.5, 13.1, np.nan, 14.0, 12.8])
temps.mean()                # nan  ← 하나라도 있으면
np.nanmean(temps)           # 13.1
np.isnan(temps).sum()       # 1
temps[~np.isnan(temps)]     # NaN 빼기
np.nan == np.nan            # False !
```

`nanmean` · `nanmax` · `nanargmax` · `nanpercentile` · `isnan`

# 실습 1 · 2 (Level 1 → Level 2)

## Level 1 따라하기

- 리스트 vs 배열 속도, 생성 · 브로드캐스팅
- 인덱싱 · 마스크 · `where` · `select`
- `axis` 예상하기, NaN 다루기

## Level 2 바꾸기

- 가중치 `[0.2, 0.5, 0.3]`으로 바꾸고 등급 변화
- "기말이 10점 이상 오른 학생" 마스크
- 평균이 가장 낮은 과목 `argmin`

# 실습 3 (Level 3): 1년치 기온 데이터

```
path = get_data("weather_daily.csv")      # 2025년 가상 데이터, 365일
data = np.genfromtxt(path, delimiter=",", skip_header=1, usecols=(1, 2, 3, 4, 5))
dates = np.genfromtxt(path, delimiter=",", skip_header=1, usecols=0, dtype=str)
avg_t, max_t, min_t, rain, humid = data.T
```

- 빈 칸 → 자동으로 `nan` (평균기온 3일 결측)
- 날짜는 문자열로 따로

# 기본 요약과 월별 요약

```
np.nanmean(avg_t)                          # 연평균
dates[np.nanargmax(max_t)]                 # 가장 더운 날
daily_range = max_t - min_t                # 일교차 (배열 연산)
(rain > 0).sum(), (max_t >= 33).sum()      # 비 온 날, 폭염일

months = np.array([int(d[5:7]) for d in dates])
for m in range(1, 13):
    sel = months == m                      # 그 달만 고르는 마스크
    print(m, np.nanmean(avg_t[sel]), rain[sel].sum())
```

# 결측 채우기와 이동평균

```
filled = avg_t.copy()
nan_idx = np.where(np.isnan(filled))[0]
ok = ~np.isnan(avg_t)
filled[nan_idx] = np.interp(nan_idx, np.where(ok)[0], avg_t[ok])   # 선형 보간

ma7 = np.convolve(filled, np.ones(7) / 7, mode="valid")   # 7일 이동평균
len(ma7)       # 365 - 7 + 1 = 359
```

이동평균 = **최근 n일 평균** → 들쭉날쭉한 값에서 추세를 본다

# 이상값: 잔차 z-점수

```
trend = np.convolve(sensor, np.ones(7) / 7, mode="same")
residual = sensor - trend                        # 추세에서 벗어난 정도
z = (residual - residual.mean()) / residual.std()
found = np.where(np.abs(z) > 3)[0]
```

- 기온은 계절 변화가 크다 → 전체 평균과 비교하면 **여름 · 겨울이 모두 이상값**
- **추세를 뺀 잔차**로 비교해야 센서 오류만 드러난다

> 데이터의 성질을 알아야 **알맞은 방법**을 고를 수 있다

# AI 코드: axis 혼동

"학생별 평균을 구하고 80점 이상 학생 이름을 골라 줘"

```
avg_ai = scores.mean(axis=0)        # shape (3,)  ← 과목별!
names[avg_ai >= 80]                 # IndexError: 길이가 맞지 않음
```

운이 나쁘면(학생 수 = 과목 수) **오류 없이 틀린 답**이 나온다

# 검증: shape와 작은 예제

```
tiny = np.array([[100, 0, 50],     # 학생 A: 평균 50
                 [ 80, 80, 80]])   # 학생 B: 평균 80
tiny.mean(axis=0)    # [90. 40. 65.]  과목별
tiny.mean(axis=1)    # [50. 80.]      학생별 ✓

student_avg = scores.mean(axis=1)
assert student_avg.shape == (scores.shape[0],)
```

1. 결과 **shape**가 의도와 맞는가
2. **손계산 가능한** 작은 예제와 일치하는가

# AI에게 배열 코드를 요청할 때

```
scores는 shape (학생 수, 과목 수)인 NumPy 배열입니다.
- 학생별 평균(결과 shape는 (학생 수,))을 구하는 코드를 주세요.
- np.nan이 있을 수 있으니 nan을 무시하는 함수를 쓰세요.
- 2×3 작은 예제로 손계산과 비교하는 assert를 함께 주세요.
```

**입력의 모양, 결과의 모양, 결측 처리**를 말로 알려 준다

# 흔한 실수 모음

| 실수 | 결과 | 해결 |
|---|---|---|
| `axis` 반대로 | 엉뚱한 집계 | shape 확인, 작은 예제 |
| 마스크에 `and` | `ValueError` | `&`, 괄호 |
| NaN 섞인 `mean()` | `nan` | `nanmean` |
| `x == np.nan` | 항상 False | `np.isnan(x)` |
| `b = a` 후 `b` 수정 | `a`도 바뀜 | `a.copy()` |
| 난수 시드 없음 | 매번 결과 다름 | `default_rng(42)` |

# 이번 주 과제 (10점)

| 문제 | 내용 | 배점 |
|---|---|---|
| 1 | `student_avg` · `subject_best` · `count_all_pass` (NaN 포함) | 2 |
| 2 | `minmax` · `zscore` 정규화 (NaN 유지, 분모 0 처리) | 2 |
| 3 | `iqr_outliers` · `clean_mean` IQR 이상값 | 3 |
| 4 | `moving_average` · `longest_rise` (AI 협업) | 3 |

**반복문 없이** NumPy 연산으로 (문제 4 `longest_rise`만 예외)

# 자기 점검

1. `np.array([1, 2]) * 2`와 `[1, 2] * 2`의 결과는?
2. shape `(5, 3)`에서 `axis=1` 평균의 shape는?
3. 60 이상 90 미만을 고르는 마스크는?
4. NaN이 섞인 배열의 평균을 구하는 방법 두 가지는?
5. 기온 이상값을 잔차로 찾는 이유는?

# 정리와 다음 주

| 오늘 | 핵심 |
|---|---|
| 배열 사고 | 반복문 대신 배열 전체 연산, 브로드캐스팅 |
| 선택 | 인덱싱 `[행, 열]`, 마스크 `& \| ~`, `where` |
| 집계 | axis = 사라지는 방향, `argmax`, `percentile` |
| 결측 | `nan` 함수들, `interp` |
| AI | shape 확인 + 손계산 예제 |

**다음 주 · Week 10** pandas로 데이터 불러오기와 탐색